

Batch Content Generation System

Product & Technical Specification

Project Goal

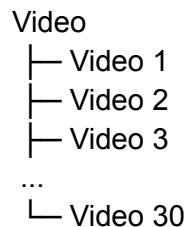
Implement a **Batch Content Generation System** within the existing React Flow (XYFlow) workflow builder.

The goal is to allow a user to upload a single video and automatically generate dozens of content variations without cluttering the workflow canvas.

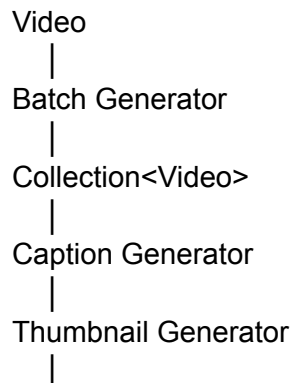
This is **not** a feature that creates 30 visual nodes. Instead, it introduces a **Collection** data type that allows one node to internally process multiple outputs.

Core Concept

Instead of:



The workflow should behave like:



Publisher

Only one Batch Generator node exists on the canvas.

Internally, that node creates multiple execution jobs.

User Experience

The user should be able to:

- Upload one video.
- Choose how many variations to generate.
- Keep the original editing style.
- Select output platform.
- Execute the workflow.
- View progress.
- Inspect every generated result.

The workflow should remain clean regardless of whether the user generates:

- 5 videos
 - 30 videos
 - 100 videos
-

New Data Type

Introduce a new workflow data type.

Collection<T>

Examples:

Collection<Video>

Collection<Image>

Collection<Text>

Collection<Audio>

Nodes must be capable of accepting either:

- Single Item
 - Collection
-

Batch Generator Node

Create a custom React Flow node named:

Batch Video Generator

Purpose:

Receive one video.

Generate multiple content variations while preserving the original video's style.

Node Inputs

- Source Video
 - Transcript
 - Video Metadata
 - Style Blueprint
-

Node Settings

Variation Count

- 5
- 10
- 20
- 30
- Custom

Platform

- TikTok

- Instagram
- YouTube Shorts
- Facebook
- X
- LinkedIn

Output Duration

- Auto
- 15 sec
- 30 sec
- 45 sec
- 60 sec

Variation Strategy

- Different Hooks
- Different CTAs
- Different Openings
- Different Story Structure
- Different Captions
- Mixed

Style Preservation

- Enabled by default

Brand Lock

- Enabled

Speaker Lock

- Enabled

Processing Pipeline

Upload Video



Video Analysis

↓

Transcript

↓

Scene Detection

↓

Editing Style Analysis

↓

Create Style Blueprint

↓

Generate Execution Jobs

↓

Batch Processing

↓

Collection<Video>

↓

Downstream Nodes

Style Blueprint

Before generating variations, analyze the original video and build a reusable blueprint.

The blueprint should capture:

- Editing rhythm
- Speaker identity
- Color grading
- Caption style

- Font usage
- Transition style
- Camera movement
- Visual pacing
- Hook style
- Tone
- Brand identity

Every generated variation should reference this blueprint.

Do not generate videos from scratch.

Generate variations from the blueprint.

Execution Model

When the user selects:

Generate 30

Create:

30 execution jobs

Example:

Job 1

Job 2

Job 3

...

Job 30

These jobs execute in parallel when resources allow.

The canvas should still display only one node.

Node UI

The Batch Video Generator node should display:

- Total requested outputs
- Progress
- Success count
- Failed count
- Running count
- View Outputs button

Example layout:

Batch Video Generator

Output:

30 Videos

Progress



18 / 30 Complete

View Outputs

Output Viewer

Selecting “View Outputs” opens a drawer or modal.

Display:

Video 1

Video 2

Video 3

...

Video 30

Each output should support:

- Preview
 - Download
 - Regenerate
 - Delete
 - Approve
-

Collection-Aware Nodes

Every downstream node should automatically iterate through collections.

Example:

Collection<Video>

↓

Caption Generator

↓

Collection<Caption>

Then:

Collection<Caption>

↓

Thumbnail Generator

↓

Collection<Thumbnail>

No manual loops should be required.

Automatic Iteration

If a node receives:

Collection<T>

The engine should automatically:

For each item

Run node

Collect result

Return Collection

The user should never have to configure iteration manually.

Example Workflow

Upload Video



Analyze Video



Batch Video Generator



Caption Generator



Thumbnail Generator



Social Caption Generator



Publisher

Future Collection Types

The architecture should support:

Collection<Image>

Collection<Podcast>

Collection<Article>

Collection<Email>

Collection<Post>

Collection<Ad>

Collection<Blog>

Collection<Carousel>

This allows the same workflow engine to power all AI content generation features.

Scalability

The execution engine should support:

- Parallel execution
- Queued execution
- Retry failed jobs
- Pause
- Resume
- Cancel
- Progress tracking

Performance Goals

The workflow canvas should remain responsive even when generating:

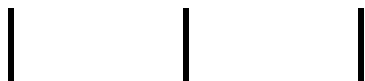
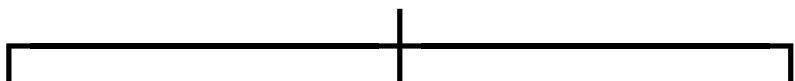
- 30 outputs
- 100 outputs
- 500 outputs

Execution complexity must never affect canvas performance.

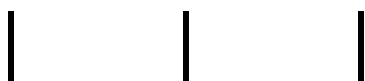
Video Input

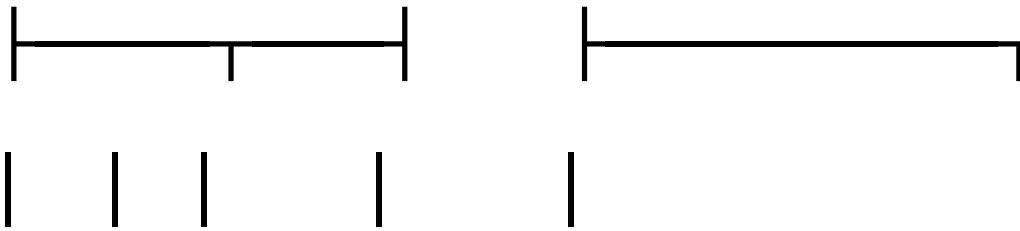


Transcribe Audio



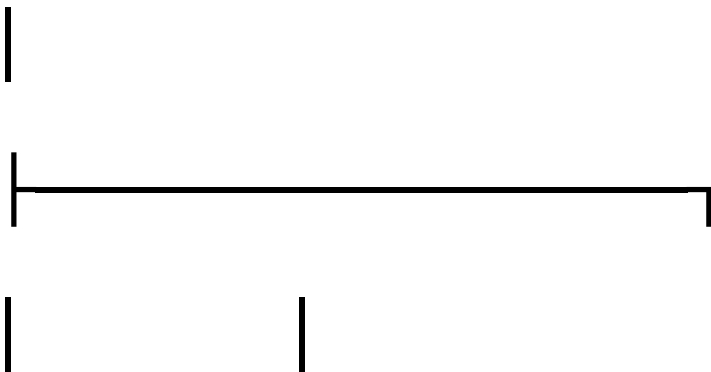
Captions Clips Summary





TikTok Reels Shorts X Thread Blog

**Instagram Reel YouTube Shorts LinkedIn
Post**



30 Hooks 30 Thumbnails



30 Titles 30 CTAs

Video

|

Batch Generate

|

Collection (30 videos)

|

Caption Node

|

Publish

[

{

```
id:1,  
  
video:"...",  
  
transcript:"..."  
  
},  
  
{  
  
id:2,  
  
video:"..."  
  
},  
  
...  
  
]  
  
UI
```

Instead of exploding the graph...
I'd show this inside ONE node.

30

Duration

15 sec

Platform

TikTok

Hook style

Aggressive

Caption Style

Storytelling

AI Temperature

0.9

Output Folder

Campaign A

One click.

30 jobs begin.

Execution Engine

Internall

Video

↓

Planner AI

↓

[
 job1,
 job2,
 job3,

...

 job30
]

↓

Worker Queue

↓

GPU Worker 1

GPU Worker 2

GPU Worker 3

GPU Worker 4



Results

Workers process in parallel.

Downstream nodes

Imagine

Generate 30 Clips



Caption Generator

The Caption node doesn't know it's receiving one clip.

It simply receives

30 clips

and automatically loops.

Result

30 captions

Next node

Thumbnail Generator



30 thumbnails

Next node
TikTok Publisher



30 scheduled uploads

The secret sauce

I'd introduce a **Collection** data type.

Instead of

Video

You now have

Collection<Video>

Or

Collection<Image>

Collection<Text>

Collection<Audio>

Supports Collections

YES

for each item

execute node

combine outputs

The user never has to build loops manually.

User experience

The creator drags just a few nodes:

Upload Video

|

Analyze

|

Generate 30 Variations

|

Generate Captions

|

Generate Thumbnails

|

Publish

They click **Run**, and behind the scenes the engine creates and processes 30 independent jobs through the entire pipeline. The canvas stays simple, while the execution engine handles all the complexity.

This pattern scales beyond videos too. The same “collection” concept can power image generation, blog creation, email campaigns, podcast snippets, or any other workflow where one input fans out into many outputs. It’s a flexible foundation for an AI automation platform rather than a feature built only for videos.

What I would build instead of their nodes

Neither n8n nor Langflow has a node exactly like what you're envisioning for AI video creation. I'd create a node that looks something like this:

🎬 AI Batch Generator

Source: Podcast.mp4

Variations: [30 ▼]

Preserve Style: ☒

Preserve Speaker: ☒

Preserve Branding: ☒

Progress18/30

Collection<Video>

👁 View Outputs

Then your workflow stays beautifully simple:

Upload Video



AI Analysis

|



 Batch Generator (30)

|



Collection<Video>

|



Caption Generator

|



Thumbnail Generator

|



Publisher

From what you've described over the past few messages, I actually **wouldn't copy any existing node 1:1**. You're combining ideas from React Flow, n8n, and AI content generation in a way I haven't seen packaged together.

If you're building this well, the **"AI Batch Generator"** node itself could become one of the signature features of your platform rather than something borrowed from another workflow tool.

Success Criteria

The implementation is complete when:

- One source video can generate multiple variations.
- The canvas displays only a single Batch Generator node.
- Variations preserve the original style using the Style Blueprint.
- Downstream nodes automatically process collections.
- Users can monitor progress.
- Users can inspect individual outputs.
- The system scales cleanly without adding duplicate nodes to the workflow.